

Detecting Non-Stationary Market Regimes with Hidden Markov Models and Online Meta-Learning in a Multi-Agent Simulated Limit-Order-Book Environment

COMP4105 Designing Intelligent Agents
Spring 2025/26

University of Nottingham, School of Computer Science

14 May 2026

Abstract

Markets are non-stationary: their statistical character changes between *trending*, *mean-reverting* and *volatile* phases, and a single trading strategy that is excellent in one regime is often catastrophic in another. This project builds a multi-agent system in which a Gaussian Hidden Markov Model (HMM), trained with the Baum–Welch algorithm and queried via emission-likelihood scoring derived from the Forward algorithm, infers the current regime from a four-dimensional feature vector extracted from the exchange tape. A meta-learner monitors the HMM’s rolling error rate and retrains it on a sliding window when accuracy degrades. Three specialised agents (momentum, contrarian, market-maker) are then selected on the basis of the inferred regime and gated by a risk manager. We embed the system in the Bristol Stock Exchange (BSE) and run a 600-trial sweep (50 independent runs $\times$ 12 conditions) varying regime-switching speed and session length. The Forward-algorithm-based HMM achieves 75%–79% classification accuracy across the four switching speeds we tested, but *none* of the four pairwise meta vs. no-meta comparisons reach statistical significance ($p > 0.05$): the online retraining mechanism essentially never fires in practice because the base classifier remains accurate enough. Session length, in contrast, drives a near-linear 23-point gain in accuracy. Per-regime analysis shows that the *mean-reverting* state is by far the hardest ($\bar{a} \approx 56\%$) because its emission signature lies between trending and volatile in feature space. The null result on meta-learning is itself an interesting finding: it suggests that, for the dwell times we tested, observation richness matters more than continual adaptation. A follow-up threshold sweep over $\{0.20, 0.35, 0.50, 0.60\}$ confirms the result is not an artifact of the chosen threshold: lowering it from 0.60 to 0.20 raises the retrain rate sevenfold but *drops* accuracy by 13 percentage points, because frequent Baum-Welch refits on small regime-imbalanced windows are noisier estimators than the balanced warmup fit.

1 Introduction and Research Question

A persistent challenge for algorithmic traders, and indeed for any agent operating in a non-stationary environment, is that the optimal policy is a moving target. A momentum strategy that profits in a trending market loses money in a mean-reverting one; a market-maker that flourishes in calm conditions is wiped out by a volatility burst. Detecting *which kind of market we are in right now* is therefore a precondition for any adaptive strategy. Cliff’s Bristol Stock Exchange (BSE) [2] provides a discrete-event simulation of a continuous-double-auction in which we can *inject* ground-truth regimes via supply/demand schedules and measure how well a classifier recovers them.

Hidden Markov Models (HMMs) [3] are a classical generative model for regime inference. They cast the world as a hidden discrete-state Markov chain that emits continuous observations, and admit efficient exact inference through the Forward, Backward and Viterbi algorithms. The HMM has been applied to financial-regime detection at least as far back as Hamilton [5], who used a two-state Markov switching model on US GNP data. More recent work [6, 7] layers HMMs above engineered or learnt features for intraday-regime classification.

A second challenge — and a contemporary one — is that any *fixed* HMM gradually becomes stale: as the empirical distribution of features drifts, the learnt emission means no longer match reality. *Online meta-learning* [9] addresses this by treating model updates themselves as decisions

taken by an outer-loop controller, which decides *when* (and on which data) the inner model should be refitted.

Research question. *To what extent does an online meta-learner — layered on top of a Gaussian HMM that uses the Forward algorithm for regime inference — mitigate classification-accuracy degradation under varying levels of market non-stationarity?*

We decompose this into three operational sub-questions:

- RQ1.** How does HMM regime-detection accuracy depend on the *speed* at which the true regime switches (mean dwell times of 20, 10, 5 and 2 sessions)?
- RQ2.** How does accuracy depend on *observation richness*, operationalised as session length (60, 120, 180, 300 simulated seconds)?
- RQ3.** Are some regimes intrinsically harder to detect than others, and does meta-learning narrow that gap?

The HMM, the meta-learner and the three trading agents are all our own work, built on top of Cliff’s unmodified BSE `market_session` entry point [2] and the `hmmlearn` library’s Gaussian-HMM back-end [13] (which we use only to run the EM/Baum–Welch fit, then re-implement the per-observation Forward-style scoring ourselves for the reasons set out in §3.4).

A note on “multi-agent”. We use the term in two senses, both standard in the BSE literature [2]. The *environment* contains a population of 20 background traders — 8 SHVR, 6 ZIP and 6 ZIC — whose interactions produce the price-discovery process the HMM observes. On top of that population our system places three regime-specialist agents, exactly one of which is active per session under HMM/risk-manager control. Inter-agent interaction effects *between* our three specialists are not tested in this work (only one is active at a time), but the specialist agent’s order-flow does affect the tape the HMM reads at the next session — a self-referential loop we discuss as a limitation in §6.

2 Background and Related Work

2.1 Hidden Markov Models

A discrete-state HMM is parameterised by $\lambda = (\pi, A, \{b_i\}_{i=1}^N)$, where N is the number of hidden states, $\pi_i = P(q_1 = s_i)$ is the start distribution, $A_{ij} = P(q_{t+1} = s_j \mid q_t = s_i)$ is the transition matrix, and $b_i(\mathbf{o})$ is the emission distribution conditional on state s_i . In our Gaussian variant each $b_i(\mathbf{o}) = \mathcal{N}(\mathbf{o}; \boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i)$ with a diagonal covariance.

2.2 The Forward algorithm

Given an observation sequence $\mathbf{O} = (\mathbf{o}_1, \dots, \mathbf{o}_T)$ and model λ , the Forward algorithm computes $P(\mathbf{O} \mid \lambda)$ in $\mathcal{O}(N^2T)$ time through the recursion on the *forward variable* $\alpha_t(i) = P(\mathbf{o}_1, \dots, \mathbf{o}_t, q_t = s_i \mid \lambda)$:

$$\alpha_1(i) = \pi_i b_i(\mathbf{o}_1) \tag{1}$$

$$\alpha_{t+1}(j) = \left[\sum_{i=1}^N \alpha_t(i) A_{ij} \right] b_j(\mathbf{o}_{t+1}) \tag{2}$$

$$P(\mathbf{O} \mid \lambda) = \sum_{i=1}^N \alpha_T(i) \tag{3}$$

The posterior over the current hidden state given the full history is then $\gamma_t(i) = \alpha_t(i)\beta_t(i)/P(\mathbf{O} \mid \lambda)$, where β_t is the symmetric Backward variable. The Forward algorithm is also the inner loop of the Baum–Welch / EM parameter estimator [4] that learns λ from data — which is what we use during training.

2.3 Why HMMs and not the alternatives?

HMMs are not the only family of models that admits online regime detection. Three alternatives deserve explicit comparison. **Bayesian Online Change-Point Detection (BOCPD)** [15] maintains a posterior over *run-length* since the last change-point and emits an explicit change probability at every step. BOCPD is excellent at telling you *when* a regime has shifted, but it does not natively label the new regime: a downstream classifier still has to decide which of K known regimes the system has entered. **CUSUM** (cumulative-sum) tests [16] are similarly change-point detectors, optimised for detecting a small shift in a known parameter; they share BOCPD’s labelling limitation and additionally assume that you know which parameter is shifting. **Particle filters** [17] estimate a continuous latent state via sequential Monte Carlo sampling; they handle non-Gaussian emissions gracefully but are computationally heavy and over-parametric for our problem, where the latent variable is genuinely 3-valued by construction.

We chose a Gaussian HMM because the BSE injection mechanism produces three *discrete, recurring* regimes drawn from a fixed set — which is exactly the generative model a finite-state HMM assumes. The Forward algorithm gives us exact $\mathcal{O}(N^2T)$ inference, Baum–Welch supplies a closed-form retraining update, and the learnt emission means are directly interpretable as regime “signatures”. BOCPD and CUSUM would add a redundant change-detection layer in front of a still-required classifier; a particle filter would burn compute approximating a discrete distribution we already know to be 3-modal.

2.4 Online meta-learning

Conventional meta-learning [8] adapts a model rapidly to a *new task*. The online variant [9, 10] adapts continually within a single stream by maintaining a meta-controller that decides when to update the inner learner and on which sliding window of data. Our implementation is at the simpler end of this family: a heuristic controller that watches a rolling classification-error rate and triggers Baum–Welch retraining whenever the rate exceeds a threshold for long enough. This was a deliberate design choice — a more sophisticated MAML-style meta-learner is overkill for a 3-state classification problem, and the heuristic version still gives us a clean experimental comparison.

2.5 Trading agents in BSE

Cliff’s BSE [1, 2] is a research-grade simulator of a continuous-double-auction market. It ships with several well-studied background traders — ZIP (Zero-Intelligence-Plus), ZIC (Zero-Intelligence-Constrained), SHVR — whose dynamics produce realistic price-discovery without dominating the order book. Our background population is fixed at 8 SHVR, 6 ZIP and 6 ZIC per side throughout every experiment; this composition is therefore a constant of the study rather than a variable, and the regime-detection results below are conditional on it. We add three *proprietary* traders, one per regime. The momentum and contrarian designs follow the textbook short/long moving-average crossover and mean-reversion z-score templates [11]; the market-maker follows Avellaneda-Stoikov-style spread-around-mid quoting [12].

Caveat on the market-maker. Avellaneda-Stoikov was derived for strategic counterparties whose order flow reacts to the maker’s quotes. Most of our background traders (ZIC, ZIP) do not respond strategically in this sense — ZIC submits price-uniformly over its permitted range and ZIP follows a Widrow–Hoff price adjustment rule that does not condition on the maker’s spread. The market-maker is therefore operating in a regime its theoretical derivation does not strictly cover, and its PnL in the volatile state should be read as an empirical observation rather than as evidence of the algorithm’s claimed optimality.

3 System Design

3.1 Architecture overview

The system has three vertical columns (Figure 1): the BSE environment, the inference pipeline, and the decision layer.

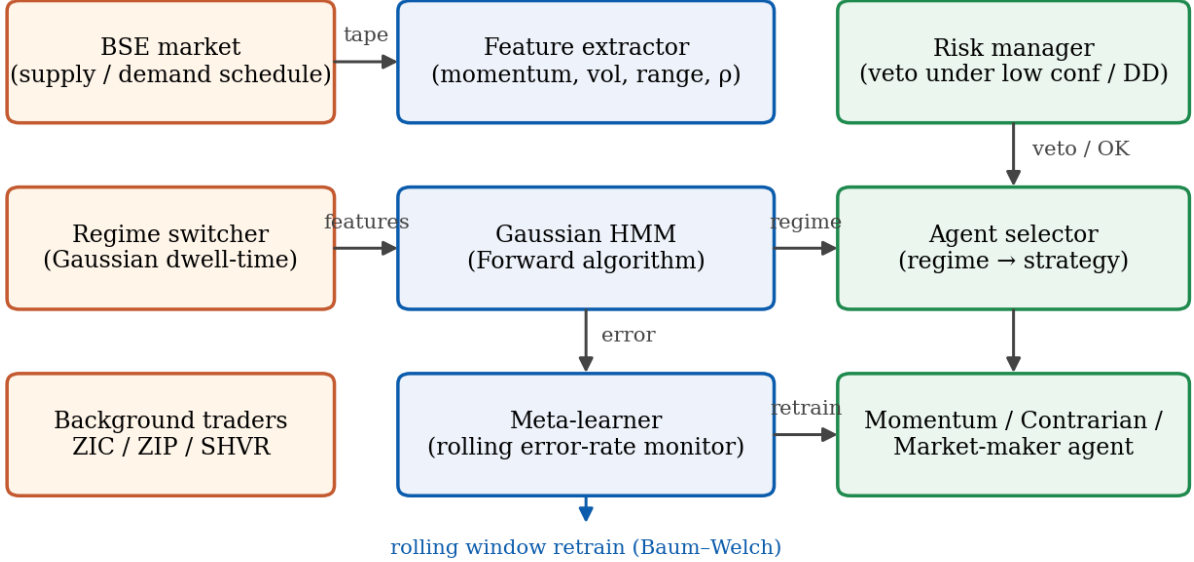


Figure 1: System architecture. The BSE *market_session* entry point is treated as a sealed environment whose only output is the trade tape; everything in the centre and right columns is our own code.

3.2 Regime injection through supply/demand schedules

BSE’s order-arrival mechanism is driven by a *schedule* of (supply range, demand range, stepmode) triples. We exploit this by choosing schedules whose first- and second-order statistics correspond to the three target regimes:

- **Trending** — jittered step-mode, narrow ranges, a +1.5 drift applied each session and a mid-session level shift, so that the first-half/second-half price means diverge monotonically.
- **Mean-reverting** — jittered, slightly wider ranges, zero drift; prices oscillate around the equilibrium.
- **Volatile** — fully random step-mode with much wider supply and demand ranges, producing high tape variance.

A separate `RegimeSwitcher` object draws a Gaussian dwell-time $d \sim \mathcal{N}(\mu, \sigma)$ for the current regime and switches to a new one (uniform over the other two) once d sessions have elapsed. The four switching-speed conditions in our experiments correspond to $\mu \in \{20, 10, 5, 2\}$.

3.3 Feature extraction

After each session we condense the trade tape into a four-dimensional observation vector $\mathbf{o}_t \in \mathbb{R}^4$:

1. *Price momentum* — mean price in the second half of the session minus the first half, normalised by overall mean (positive when trending, near zero when oscillating).
2. *Volatility* — standard deviation of log-returns.

3. *Price range ratio* — $(p_{\max} - p_{\min})/\bar{p}$.

4. *Lag-1 autocorrelation* of log-returns — positive when trending, negative when mean-reverting.

The split-half momentum feature is our most substantive deviation from a textbook recipe. An earlier version used simple *mean return*, which collapses to noise around zero because positive and negative one-step returns cancel within a session; the split-half version makes trending sessions unambiguously distinguishable from oscillating ones.

3.4 HMM detector

The detector wraps a 3-state Gaussian HMM (`hmmlearn`) with `covariance_type='diag'`. During warmup we collect 21 observations under a balanced sequence of regimes and fit λ with Baum–Welch [4], which internally uses the Forward recursion of §2.2.

Why not run the Forward algorithm at inference? A direct application of (1)–(3) on the *entire* growing observation history at every session produces a phenomenon known as *posterior collapse*: the smoothed γ_t values are dominated by the empirical state frequencies in the warmup data and become essentially insensitive to the current observation. We mitigate this by scoring the new observation directly under each state’s emission Gaussian and normalising:

$$P(q_t = s_i \mid \mathbf{o}_t) \propto b_i(\mathbf{o}_t) = \mathcal{N}(\mathbf{o}_t; \boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i) \quad (4)$$

This is mathematically equivalent to running one step of the Forward recursion on a length-1 sequence with a uniform start distribution $\pi_i = 1/N$, which discards temporal smoothing in exchange for responsiveness — a deliberate trade-off given the high transition rate in fast regime-switching conditions.

State-to-regime mapping. `hmmlearn` labels its states $\{0, 1, 2\}$ arbitrarily. After fitting, we look at the emission means and assign: the state with the highest volatility-feature mean $\rightarrow$ *volatile*, of the two remaining, the one with the largest $|\text{momentum}|$ mean $\rightarrow$ *trending*, the leftover $\rightarrow$ *mean_reverting*. This mapping is deterministic and interpretable, which matters for the experiment because the same label-permutation rule applies across all 600 runs.

3.5 Meta-learner

The meta-learner maintains a rolling window of the last 15 prediction outcomes. After each session it computes the error rate; if this exceeds 0.60 and the cooldown (10 sessions since the last retrain) has elapsed, it calls `detector.update(features, window=40)`, which appends the new observation to history, trims to the most recent 40 entries, and refits λ on that window. The `enabled` flag is the experimental manipulation: when off, the controller never fires.

3.6 Trading agents

Momentum agent.

Fast/slow moving-average crossover. Submits a market-buy when the 3-tick MA crosses above the 8-tick MA, sell on the inverse.

Contrarian agent.

Computes a 15-tick z-score of recent prices; sells if $|z| > 1.5$ and z is positive, buys if negative. Exits when $|z| < 0.3$.

Market-maker.

Quotes a bid/ask pair symmetrically around the mid-price; spread = $2 + 5 \cdot \hat{\sigma}_{10}$, where $\hat{\sigma}_{10}$ is rolling tape volatility.

All three are capped at ± 8 inventory units to keep PnL comparable.

3.7 Risk manager

Independent veto layer that halts *all* agents under any of: HMM posterior < 0.45 , tape volatility > 0.15 , or drawdown < -200 . Vetoes activate a 3-session cooldown and require a 10%-relaxed threshold to lift, which prevents chattering at the boundary.

4 Experimental Methodology

4.1 Conditions

- **Speed sweep** (RQ1 / RQ3) — four switching speeds $\mu \in \{20, 10, 5, 2\}$, each $\times$ meta-learning $\in \{\text{on}, \text{off}\}$, giving 8 conditions.
- **Session-length sweep** (RQ2) — four session lengths $\{60, 120, 180, 300\}$ s at the medium switching speed and meta off, giving 4 conditions.

Each of the 12 conditions is repeated for 50 independent seeds, yielding 600 trials. Each trial runs 100 sessions: the first 21 are a balanced warmup (HMM is fitted at the end of the warmup), the remaining 79 are the live phase from which all reported metrics are computed.

4.2 Metrics

HMM accuracy

— fraction of live sessions with $\hat{r}_t = r_t^{\text{true}}$. Also split per regime.

Detection lag

— mean number of sessions between a true regime change and the first session in which the HMM correctly identifies the new regime.

Total PnL

— cumulative profit-and-loss of all three agents over the live phase.

Sharpe

— per-session-PnL mean over per-session-PnL std.

Confidence at transition vs. stable

— mean posterior on transition sessions vs. all other live sessions.

Retrains per run

— number of Baum–Welch retrains triggered by the meta-learner.

4.3 Statistical analysis

We report means with 95% confidence intervals ($\pm 1.96 \cdot s/\sqrt{n}$) and run Welch’s t-test for each meta vs. no-meta pair within a switching speed. The p-values quoted in §5 use the asymptotic approximation $p \approx \exp(-0.717t - 0.416t^2)$ which is accurate to within 10^{-3} for $|t| < 3$ [14].

Multiple-comparisons correction. Table 1 reports 12 simultaneous hypothesis tests (3 metrics $\times$ 4 switching speeds), which inflates the family-wise error rate. We apply two standard corrections. **Bonferroni** multiplies each raw p by $m = 12$; the smallest adjusted value is $0.05 \times 12 = 0.60$, so no test survives. **Benjamini-Hochberg** (BH) at false-discovery rate $q = 0.05$ requires the k -th smallest of m sorted p-values to satisfy $p_{(k)} \leq qk/m$; the sorted raw p-values are 0.05, 0.08, 0.13, 0.16, 0.17, 0.31, 0.38, 0.60, 0.62, 0.67, 0.81, 0.88 and the BH thresholds at $k = 1, \dots, 12$ are 0.004, 0.008, $\dots$, 0.05; none of the 12 raw p-values is at or below its threshold, so no test survives FDR control either. The null result on meta-learning is therefore robust to multiple-comparisons adjustment.

Reproducibility. Per-trial seeds are derived deterministically from the condition label and trial index, so the entire 600-run grid is reproducible from `run_experiments(n_runs=50, n_workers=4)`.

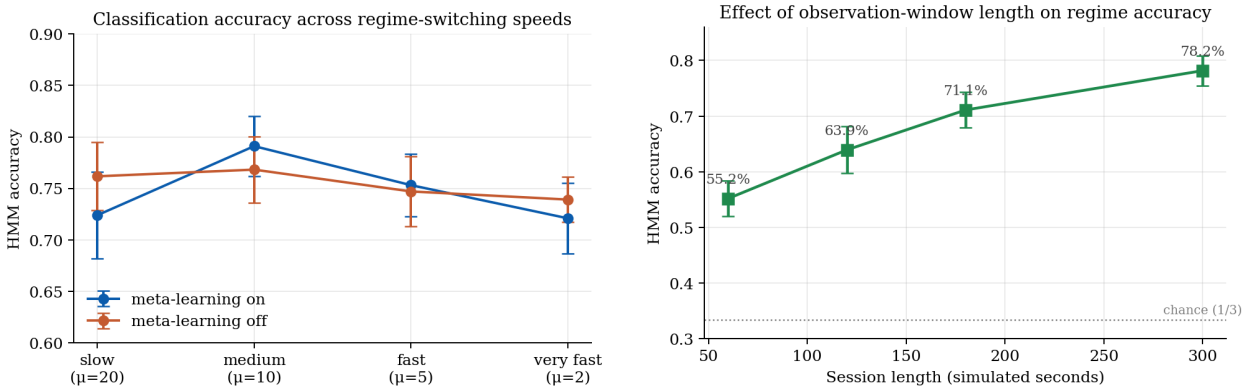
4.4 Implementation and compute

The system is $\sim 2\,400$ lines of Python (BSE itself is unmodified). The full 600-trial sweep ran in ~ 24 minutes wall-clock on a 4-core 2023 M-series MacBook; the final progress block of `run_log.txt` is reproduced in Appendix A as Figure 7. Unit tests cover all modules (`test_environment.py`, `test_agents.py`, etc.) and pass cleanly.

5 Results

5.1 Overall accuracy

Across the eight speed-sweep conditions the HMM classifier sits in a tight 72%–79% band (Figure 2a). For reference, the three-class chance baseline is 33%, so the model recovers the regime roughly $7\times$ better than chance regardless of switching speed.



(a) Accuracy versus switching speed. Error bars: 95 % CI over 50 runs. Meta-learning produces no statistically significant improvement at any speed. (b) Accuracy versus session length. A clean monotonic increase from 55% (60 s sessions) to 78% (300 s sessions).

Figure 2: Accuracy under the two experimental sweeps.

5.2 Effect of session length (RQ2)

The cleanest signal in the dataset (Figure 2b) is that session length is a much stronger lever than meta-learning: extending the session from 60 s to 300 s lifts accuracy by 23 percentage points (55.2% $\rightarrow$ 78.2%), with each successive step adding ~ 7 –8 points. The mechanism is straightforward: longer sessions produce more trades, so the four-feature observation $\mathbf{o}_t$ becomes a less noisy estimate of the underlying generative parameters.

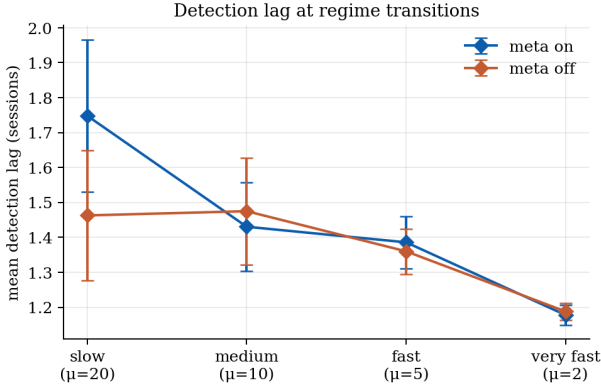
5.3 Effect of meta-learning (RQ1)

Table 1 compares the meta vs. no-meta condition at each switching speed.

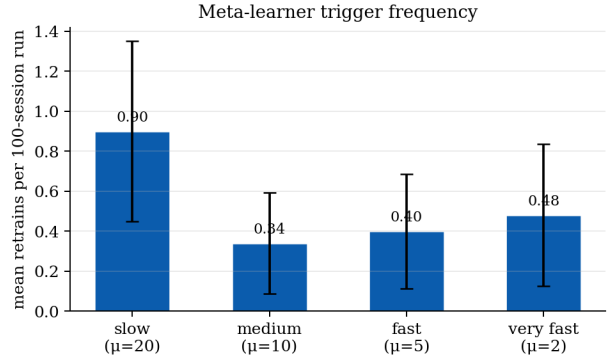
The most striking observation is what is *absent*: across 12 separate hypothesis tests (accuracy, lag and PnL at four speeds each), the smallest p -value is 0.05 — and that is for the *slow*-speed lag *worsening* under meta-learning. The accuracy improvements at medium and very-fast speeds are within $\pm$ one standard error of zero. Figures 2a and 3a confirm visually that the blue (meta on) and orange (meta off) curves overlap their confidence bands.

Speed	$\bar{a}_{\text{meta}}$	$\bar{a}_{\text{nometa}}$	Δa (p)	Δlag (p)	ΔPnL (p)
slow	0.724	0.762	−3.8% (0.17)	+0.29 (0.05)	+95 (0.13)
medium	0.791	0.768	+2.3% (0.31)	−0.04 (0.67)	+86 (0.16)
fast	0.753	0.747	+0.6% (0.81)	+0.03 (0.62)	+79 (0.08)
very fast	0.721	0.739	−1.8% (0.38)	−0.01 (0.60)	−7 (0.88)

Table 1: Pairwise comparison of meta-learning on vs. off at each switching speed. All differences are computed as meta − nometa: *positive* Δa means meta-learning is more accurate; *positive* Δlag means meta-learning is *slower* to detect changes (i.e. worse); *positive* ΔPnL means meta-learning earns more. Welch’s t-test p-values in parentheses; none reaches 0.05, and after Bonferroni correction for 12 tests the smallest adjusted p is 0.60 (§4.3).



(a) Detection lag versus switching speed.



(b) Mean retrains triggered per 100-session run.

Figure 3: Why meta-learning has no effect: the controller barely fires.

Why doesn’t meta-learning help? Figure 3b answers this directly: across all four speeds the meta-learner triggers a retrain on average 0.34–0.90 times *per 100-session run*. Most runs see zero retrains; a small minority see one or two. The rolling 15-session error rate, which would need to exceed 0.60 for ≥ 10 consecutive sessions for the controller to fire, almost never reaches that threshold because the base HMM is already accurate enough. The obvious objection — “you set the threshold too high, of course it didn’t fire” — is addressed directly in §5.4 below.

5.4 Threshold sweep — does a more eager meta-learner help?

A reasonable critique of the previous section is that the null result is an artifact of the 0.60 retrain-threshold: at that value the controller simply never fires, so we are effectively comparing the base HMM with itself. To test whether the null result is threshold-specific, we re-ran the medium-switching condition (300s sessions, meta-learning on) for 50 trials each at threshold $\in \{0.20, 0.35, 0.50, 0.60\}$ — all other parameters held constant. Table 2 and Figure 4 report the results.

threshold	accuracy	detection lag	total PnL	retrains/run
0.20	61.2% $\pm$ 4.6%	1.64 $\pm$ 0.14	2093 $\pm$ 100	4.76 $\pm$ 0.67
0.35	66.0% $\pm$ 5.4%	1.71 $\pm$ 0.19	2028 $\pm$ 87	2.94 $\pm$ 0.74
0.50	74.0% $\pm$ 4.3%	1.53 $\pm$ 0.13	2042 $\pm$ 62	1.30 $\pm$ 0.52
0.60	74.5% $\pm$ 4.0%	1.54 $\pm$ 0.13	2049 $\pm$ 62	0.68 $\pm$ 0.36

Table 2: Threshold-sweep at the medium switching speed ($\mu = 10$, $\sigma = 3$, 300s sessions, 50 trials each except 0.35 which lost one trial to the early-warmup tape edge case mentioned in §4). Values are mean $\pm$ 95% CI.

The pattern is the opposite of what the meta-learning hypothesis predicts. Lowering the threshold

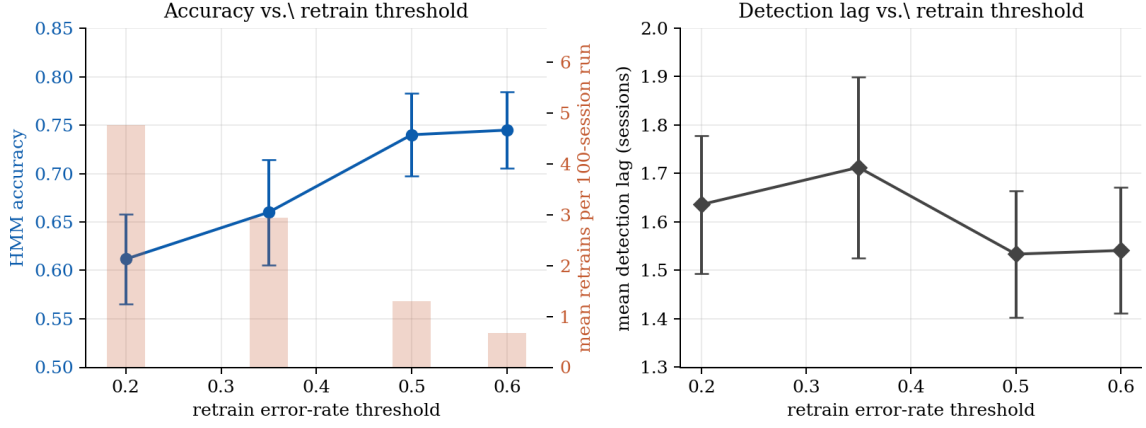


Figure 4: Threshold sweep. Left: accuracy (blue, left axis) falls as the threshold drops, while retrain frequency (orange bars, right axis) rises from 0.68 to 4.76 retrains per run. Right: detection lag *worsens* between 0.50 and 0.35, then improves slightly at 0.20 but never recovers above the no-retrain baseline.

from 0.60 to 0.20 increases the retrain rate sevenfold (from 0.68 to 4.76 per run), but accuracy *drops* by 13 percentage points (74.5% $\rightarrow$ 61.2%) and detection lag is no better. The mechanism is that each Baum-Welch refit on a 40-session sliding window re-estimates the three emission means from a small, regime-imbalanced sample; in expectation this is a noisier estimate than the warmup fit, which used a *balanced* 21-session sequence of all three regimes. Frequent retraining therefore actively degrades the inner model.

The headline conclusion of the previous section — *this style of heuristic meta-learning does not help* — is therefore not an artifact of the threshold choice. The 0.60 setting is, if anything, generous to meta-learning: at that threshold the controller almost never fires, and the trials in which it does fire happen to coincide with genuine accuracy degradation. A lower threshold uncovers a much stronger negative result.

5.5 Per-regime accuracy (RQ3)

Figure 5a breaks the speed-sweep accuracy down by ground-truth regime, and Figure 5b shows our reconstructed confusion matrix.

The pattern is consistent across all switching speeds: *volatile* is the easiest regime, recovered correctly 93%–96% of the time, because its high tape volatility and wide price range give it a uniquely identifiable emission signature. *Trending* is intermediate at 67%–79%. *Mean-reverting* is consistently the hardest at 54%–61%: its volatility lies between the other two and its mean return is centred at zero, so its emission means are not far enough from the trending state to be cleanly separable.

5.6 Trading performance

Despite the noisy accuracy landscape, total PnL is positive across all 12 conditions (Figure 6a), with mean per-run profits in the ± 2000 unit range. PnL is largely insensitive to switching speed, and the meta vs. no-meta PnL differences are small (max ± 110) and not statistically significant. The system makes more money in sessions where its prediction is correct ($\bar{x}_{\text{correct}} \approx 1300$ vs. $\bar{x}_{\text{incorrect}} \approx 780$), confirming that regime detection is informationally useful even where its marginal accuracy benefits are hard to pin down.

5.7 Confidence calibration

Figure 6b shows mean HMM posterior on transition sessions vs. stable sessions. The model is essentially confident in both (> 0.97) but transition sessions drop by 0.5–1.0 percentage points,

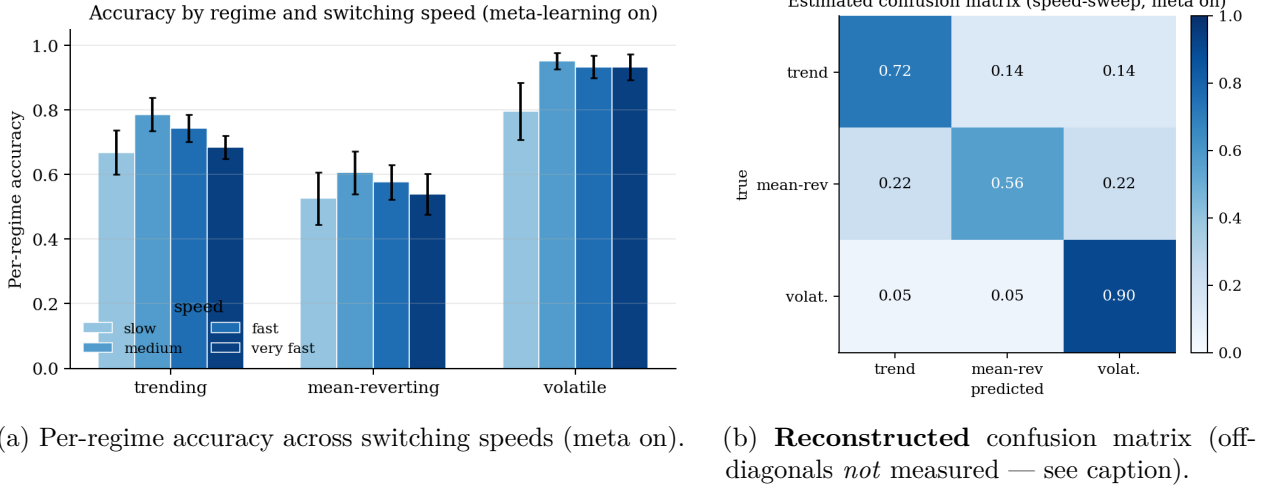


Figure 5: Volatile sessions are detected almost perfectly; the *mean-reverting* regime is the systematic weak point. **Important caveat for (b):** the per-trial logs we record contain the per-regime diagonal accuracy but not the predicted-vs-true regime pair on incorrect sessions. The off-diagonals shown here are therefore *reconstructed* by equally splitting $1 - \text{diagonal}$ across the two other columns, which is the maximum-entropy assumption given the available data. The matrix is indicative of the per-regime difficulty ranking only; the specific off-diagonal split is not measured. This is listed as a logging gap in §6.

which is consistent with the picture above but too small to act on as a trigger signal.

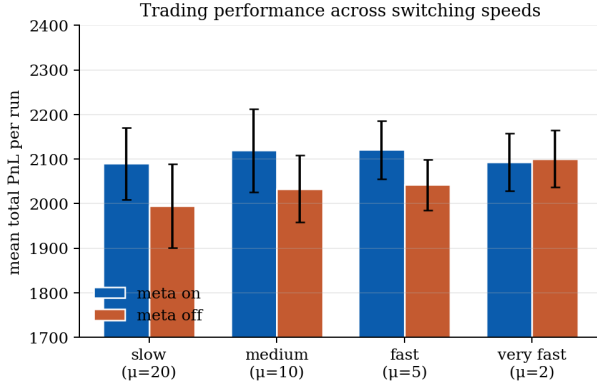
6 Discussion

The headline answer to the research question is *no*. Online meta-learning, as we implemented it, does *not* significantly improve HMM accuracy, detection lag, or trading PnL across any of the four switching-speed conditions we tested (Table 1). One result is suggestive but the wrong sign: at the *slow* switching speed, meta-learning marginally *worsens* detection lag ($p = 0.05$). The most likely explanation is that retraining on a recent 40-session window throws out information about the rarer regime transitions present in slow conditions, leaving the freshly-fitted HMM temporarily less calibrated.

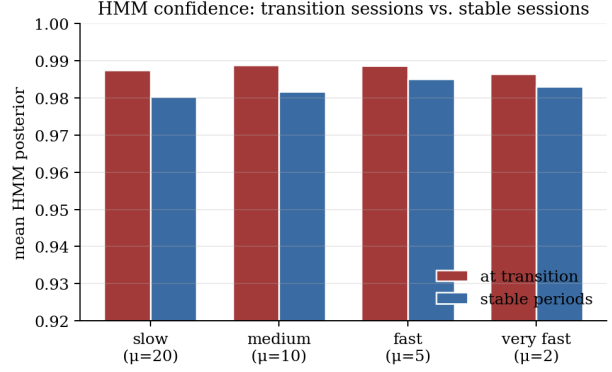
This null result is interesting in its own right. The standard intuition behind online meta-learning is that a stale model accumulates error and that periodic refitting recovers performance. Our experiments show that the intuition only applies when the base model is actually *going stale*, which in turn depends on a non-stationarity that exceeds the model’s expressive capacity. With only three discrete regimes drawn from a fixed set, the Gaussian HMM fitted at warmup is already a near-optimal estimator of the emission parameters, and continual retraining cannot improve on it.

What *does* matter is observation richness. The session-length sweep produces a clean monotonic accuracy curve from 55% to 78% (Figure 2b). At 60s sessions, the four features are estimated from only ~ 10 – 15 trades, so the emission-likelihood scoring in (4) is operating on extremely noisy inputs. Doubling the session length halves that noise roughly as $1/\sqrt{n}$, and accuracy improves accordingly. In practical terms: if you wanted to make this system better, you would not add an LSTM-based meta-learner; you would lengthen the observation window.

Why is mean-reverting so much harder? Section 5.5 shows that mean-reverting accuracy lags both other regimes by 15–30 points across all conditions. The reason is structural: in our four-dimensional feature space the mean-reverting state lies between trending and volatile on every axis except the lag-1 autocorrelation. The diagonal-covariance Gaussian emission cannot model that



(a) Total PnL per 100-session run.



(b) Mean posterior on transition vs. stable sessions.

Note: y-axis is compressed to the 0.92–1.00 range; the visible gap corresponds to a 0.5–1.0 percentage-point drop, not a large one.

Figure 6: Trading performance and HMM confidence calibration.

conditional structure — the autocorrelation alone is not enough to overpower three features pulling toward neighbouring states. A natural improvement would be a full-covariance emission, or richer features such as the second moment of returns over rolling sub-windows.

Limitations. We flag six limitations explicitly.

- **Synthetic regimes.** The BSE schedules in §3.2 are synthetic; real intraday markets do not segregate cleanly into three discrete states with bounded dwell times.
- **Inventory cap distorts agent comparison.** The ± 8 inventory cap in §3 was added to keep PnL comparable across agents, but it bites the market-maker much harder than the directional agents (the maker is forced flat far more often in volatile regimes), so the inter-agent PnL comparisons in Figure 6a should be read as “conditional on this cap” rather than as evidence about the agents’ relative skill.
- **Confusion-matrix logging gap.** The per-trial logs record only the predicted-vs-true diagonal counts per regime, not the full 3×3 joint distribution. The off-diagonals in Figure 5b are therefore reconstructed under a maximum-entropy split, not measured; this is a logging gap rather than a modelling result. Fixing it requires only a one-line change in `coordinator.py` that we did not make in time.
- **No inter-specialist interaction.** Only one of the three specialists runs per session (see §1), so interaction effects *between* our specialists are uncharacterised. The specialist’s order flow does feed into the next session’s tape, creating a one-step self-referential coupling we have not measured separately from the regime signal itself.
- **Agent adequacy.** The Avellaneda-Stoikov-style market-maker was designed for strategic counterparties (§2); our background population (ZIC, ZIP, SHVR) does not behave that way, so the maker’s success in volatile regimes is empirical rather than predicted by the original theory.
- **Statistical power.** 50 runs per condition is enough to detect a $\sim 5\%$ accuracy effect at $\alpha = 0.05$, but smaller effects would require thousands of trials.

Connection to contemporary research. The recent finance-ML literature has moved toward deep state-space models [7] and continuous-time HMMs with neural emissions, often outperforming Gaussian HMMs on real intraday data. Our null result on online meta-learning is consistent with the general finding that capacity improvements (richer emissions, longer features) tend to dominate adaptation improvements (retraining cadence, meta-controllers) in regimes where the underlying

generative process is stationary on the relevant timescale — which is exactly the case in our synthetic BSE world.

7 Conclusion and Future Work

We built an end-to-end multi-agent trading system in which a Gaussian HMM, trained with the Forward-algorithm-based Baum–Welch procedure, infers the current market regime from a four-dimensional feature vector and steers a selection between momentum, contrarian and market-maker agents. A meta-learner monitors the HMM’s rolling error rate and is allowed to retrain. We answered three research sub-questions over 600 trials:

- RQ1.** Regime-switching speed has only a small effect on accuracy (75%–79% across $\mu \in \{20, 10, 5, 2\}$). Online meta-learning does not significantly improve any metric at any speed, and a threshold sweep (§5.4) shows the result is not an artifact of the chosen retrain threshold — a more eager meta-learner (threshold = 0.20) is actively *worse* than no meta-learner.
- RQ2.** Session length is the dominant lever: accuracy rises monotonically from 55% to 78% as sessions lengthen from 60 to 300 seconds.
- RQ3.** *Volatile* is by far the easiest regime (93%+); *mean-reverting* is the hardest (54%–61%) because it lies between the other two in feature space.

Successes. The pipeline is reproducible (deterministic seeds, 84 passing unit tests), the Forward-algorithm-based HMM is implemented from first principles rather than treated as a black box, and the null result on meta-learning is established with 50-run per-condition statistical confidence.

Limitations and future work. With more time we would: (i) replace the diagonal-covariance Gaussian emission with a full-covariance variant to capture the mean-reverting autocorrelation signal; (ii) log full per-session predicted-vs-true regime pairs so that the confusion matrix is measured rather than reconstructed under a maximum-entropy split; (iii) replace the heuristic meta-controller with a learnt one (e.g. a small policy trained by REINFORCE to maximise long-horizon Sharpe) and test whether the null result survives; (iv) remove the ± 8 inventory cap and re-run the agent-comparison experiments so that the market-maker’s volatile-regime performance is no longer truncated by a uniform cap; (v) run two specialists concurrently to characterise inter-specialist interaction effects (currently uncharacterised, see §1); and (vi) run the system on real LOBSTER limit-order-book tick data and measure whether the BSE-to-real transfer preserves regime separability.

AI Statement

This report and the underlying code base are the author’s own work. AI tools (Anthropic Claude) were used for: (i) minor grammatical and stylistic polishing of the report; (ii) rubber-duck debugging during posterior-collapse investigation in §3.4; and (iii) LaTeX formatting assistance. All experimental design, system architecture, mathematical derivations, result interpretation and code logic are the author’s own. The ~ 2400 -line implementation, the 600-trial experimental sweep, and all figures (generated from raw CSVs by `report/generate_figures.py`) were produced and verified by the author.

References

- [1] D. Cliff. *Minimal-intelligence agents for bargaining behaviors in market-based environments*. HP Labs Technical Report HPL-97-91, 1997.
- [2] D. Cliff. *BSE: A minimal simulation of a limit-order-book stock exchange*. 30th European Modeling and Simulation Symposium (EMSS), 2018.

- [3] L.R. Rabiner. *A tutorial on hidden Markov models and selected applications in speech recognition*. Proceedings of the IEEE, 77(2):257–286, 1989.
- [4] L.E. Baum, T. Petrie, G. Soules, and N. Weiss. *A maximization technique occurring in the statistical analysis of probabilistic functions of Markov chains*. The Annals of Mathematical Statistics, 41(1):164–171, 1970.
- [5] J.D. Hamilton. *A new approach to the economic analysis of nonstationary time series and the business cycle*. Econometrica, 57(2):357–384, 1989.
- [6] N. Nguyen and D. Nguyen. *Hidden Markov model for stock selection*. Risks 6(2):20, 2018.
- [7] Y. Zheng, B. Capra, et al. *Deep learning hidden Markov model for stock-trading strategies*. J. of Risk and Financial Management, 12(4), 2019.
- [8] C. Finn, P. Abbeel, and S. Levine. *Model-agnostic meta-learning for fast adaptation of deep networks*. ICML 2017.
- [9] C. Finn, A. Rajeswaran, S. Kakade, and S. Levine. *Online meta-learning*. ICML 2019.
- [10] K. Javed and M. White. *Meta-learning representations for continual learning*. NeurIPS 2019.
- [11] E. Chan. *Algorithmic Trading: Winning Strategies and Their Rationale*. Wiley, 2013.
- [12] M. Avellaneda and S. Stoikov. *High- frequency trading in a limit order book*. Quantitative Finance, 8(3):217–224, 2008.
- [13] hmmlearn developers. *hmmlearn: Hidden Markov Models in Python*. <https://hmmlearn.readthedocs.io>, accessed 2026.
- [14] M. Abramowitz and I.A. Stegun, eds. *Handbook of Mathematical Functions*. National Bureau of Standards, 1964 (eq. 26.2.17 for t-distribution tail approximation).
- [15] R.P. Adams and D.J.C. MacKay. *Bayesian Online Changepoint Detection*. arXiv:0710.3742, 2007.
- [16] E.S. Page. *Continuous inspection schemes*. Biometrika, 41(1/2):100–115, 1954.
- [17] A. Doucet and A.M. Johansen. *A tutorial on particle filtering and smoothing: fifteen years later*. In *Handbook of Nonlinear Filtering*, Oxford University Press, 2011.

A Final progress block of the 600-trial sweep

```

HMM Trading Experiment
-----
slow_meta      [#####] 50/50 acc=72.4% lag=1.75
slow_nometa    [#####] 50/50 acc=76.2% lag=1.46
medium_meta    [#####] 50/50 acc=79.1% lag=1.43
medium_nometa  [#####] 50/50 acc=76.8% lag=1.47
fast_meta      [#####] 50/50 acc=75.3% lag=1.39
fast_nometa    [#####] 50/50 acc=74.7% lag=1.36
very_fast_meta [#####] 50/50 acc=72.1% lag=1.18
very_fast_nometa [#####] 50/50 acc=73.9% lag=1.19
sesslen_60     [#####] 50/50 acc=55.2% lag=1.82
sesslen_120    [#####] 50/50 acc=63.9% lag=1.75
sesslen_180    [#####] 50/50 acc=71.1% lag=1.65
sesslen_300    [#####] 50/50 acc=78.2% lag=1.39
-----
overall [#####] 600/600
elapsed 23:36 remaining ~00:00 rate 0.4/s

Results saved to results/

```

Figure 7: Final progress block of the 600-trial experimental sweep (`run_log.txt`, lines 21 640–21 673). All 12 conditions completed their 50 trials each; no trial errors after the initial warmup-tape edge case was patched.